

DOCUMENTACIÓN ACTUALIZADA – PROYECTO FINAL SUPERMERCADO

API Principal + API E-Commerce

1. Gustavo Sandoval API Principal
2. Edin Adolfo Martinez Sandoval API E-Commerce

DESCRIPCIÓN GENERAL

El proyecto consiste en el desarrollo de una solución moderna para un sistema de supermercado orientado a mejorar la administración de productos, clientes, ventas y compras.

La solución está compuesta por dos APIs desarrolladas en ASP.NET Core:

API Principal

Encargada de manejar toda la lógica principal del negocio:

- Productos
- Clientes
- Proveedores
- Compras
- Ventas
- Inventario
- Estados de venta
- Reportes

API E-Commerce

Encargada de la interacción con los usuarios:

- Catálogo de productos
- Paginación y filtros
- Carrito de compras
- Procesamiento de compra
- Integración futura con la API principal
- Pruebas mediante Postman

TECNOLOGÍAS UTILIZADAS

Tecnología	Uso
C#	Lenguaje principal
ASP.NET Core Web API	Desarrollo de APIs
Entity Framework Core	ORM y acceso a datos
SQL Server	Base de datos
Swagger	Documentación y pruebas
Postman	Pruebas de API E-Commerce
LINQ	Consultas y reportes
.NET 8	Framework principal

ARQUITECTURA GENERAL

El flujo general del sistema funciona de la siguiente forma:

Cliente/Postman/Swagger

↓

Controllers

↓

Entity Framework Core

↓

SQL Server / Datos Mock

La API E-Commerce actualmente utiliza datos simulados (Mock) para pruebas locales y posteriormente podrá conectarse directamente a la API Principal.

ESTRUCTURA DEL PROYECTO

API Principal

- Controllers/
- Models/
- Data/
- DTOs/
- Migrations/

- Program.cs
- appsettings.json

API E-Commerce

- Controllers/
- Models/
- Data/
- Program.cs
- appsettings.json
- Swagger
- Postman

BASE DE DATOS

Base utilizada

SupermercadoDB

Tablas principales

Productos

Contiene:

- Id
- Nombre
- Descripcion
- Precio
- Stock

Cientes

Contiene:

- Id
- Nombre
- Email
- Telefono
-

Proveedores

Contiene:

- Id
- Nombre
- Contacto
- Telefono

Compras

Representa compras realizadas a proveedores.

Incluye:

- Fecha
- Total
- Relación con proveedor
- Detalles de compra

Ventas

Representa ventas realizadas a clientes.

Incluye:

- Fecha
- Total
- Estado
- Relación con cliente
- Detalles de venta

FUNCIONALIDADES IMPLEMENTADAS – API PRINCIPAL

1. CRUD Productos

Endpoints:

- GET /api/productos
- GET /api/productos/{id}
- POST /api/productos
- PUT /api/productos/{id}

- DELETE /api/productos/{id}

Permite:

- Crear productos
- Editar productos
- Eliminar productos
- Consultar productos

2. CRUD Clientes

Endpoints:

- GET /api/clientes
- GET /api/clientes/{id}
- POST /api/clientes
- PUT /api/clientes/{id}
- DELETE /api/clientes/{id}

3. CRUD Proveedores

Endpoints:

- GET /api/proveedores
- GET /api/proveedores/{id}
- POST /api/proveedores
- PUT /api/proveedores/{id}
- DELETE /api/proveedores/{id}

4. Registro de Compras

Endpoint:

- POST /api/compras

Funcionalidad:

- Registra compras a proveedores
- Actualiza inventario automáticamente

- Aumenta stock de productos
- Calcula subtotales y total

5. Registro de Ventas

Endpoint:

- POST /api/ventas

Funcionalidad:

- Registra ventas a clientes
- Verifica stock disponible
- Descuenta inventario automáticamente
- Calcula subtotales y total

6. Estados de Venta

Endpoint:

- PUT /api/ventas/{id}/estado

Estados posibles:

- Pendiente
- Procesando
- Enviado
- Entregado
- Cancelado

Objetivo:

Permitir integración futura con Azure Queue Storage y notificaciones automáticas.

7. Reportes

Ventas por Fecha

Endpoint:

- GET /api/reportes/ventas-por-fecha

Ventas por Cliente

Endpoint:

- GET /api/reportes/ventas-por-cliente/{clienteId}

Compras por Proveedor

Endpoint:

- GET /api/reportes/compras-por-proveedor/{proveedorId}

Productos Más Vendidos

Endpoint:

- GET /api/reportes/productos-mas-vendidos

FUNCIONALIDADES IMPLEMENTADAS – API E-COMMERCE

La API E-Commerce fue desarrollada para simular la interacción de los usuarios con el sistema.

Actualmente utiliza datos Mock para pruebas locales.

Catálogo de Productos

Endpoint:

- GET /api/productos

Funcionalidad:

- Mostrar catálogo de productos
- Consultar productos disponibles
- Mostrar nombre, precio y stock

Productos simulados:

- Arroz
- Leche
- CocaCola
- Galletas
- Jabon
- Servilletas
- Cafe
- Aceite
- Agua

- Suavitel

Paginación y Filtros

Endpoint:

- GET /api/productos?pagina=1&cantidad=2

Funcionalidad:

- Mostrar productos por páginas
- Limitar cantidad de resultados
- Filtrar productos

Ejemplo:

/api/productos?pagina=1&cantidad=5

CARRITO DE COMPRAS

Agregar productos al carrito

Endpoint:

- POST /api/carrito/agregar/{id}

Parámetros:

- id → id del producto
- cantidad → cantidad deseada

Ejemplo:

POST /api/carrito/agregar/3?cantidad=2

Funcionalidad:

- Busca producto
- Agrega producto al carrito
- Guarda nombre, cantidad y precio

Ver carrito

Endpoint:

- GET /api/carrito

Funcionalidad:

- Mostrar productos agregados
- Mostrar cantidades
- Mostrar precios

Ejemplo de respuesta:

```
[  
{  
  "productoid": 3,  
  "nombre": "CocaCola",  
  "cantidad": 2,  
  "precio": 15  
}]
```

Procesar Compra

Endpoint:

- POST /api/carrito/comprar

Funcionalidad:

- Calcula total
- Simula compra
- Limpia carrito

Ejemplo de respuesta:

```
{  
  "mensaje": "Compra realizada",  
  "total": 30  
}
```

SWAGGER

Swagger se utiliza para:

- Probar endpoints
- Visualizar documentación
- Simular operaciones

- Validar respuestas

URL local:

<https://localhost:7239/swagger>

POSTMAN

Postman fue utilizado para probar la API E-Commerce.

Pruebas realizadas:

- GET productos
- GET carrito
- POST agregar productos
- POST comprar
- Validación de respuestas JSON

Ejemplo probado:

GET <http://localhost:5078/api/Productos>

POST <http://localhost:5078/api/Carrito/agregar/3?cantidad=2>

GET <http://localhost:5078/api/Carrito>

POST <http://localhost:5078/api/Carrito/comprar>

INVENTARIO AUTOMÁTICO

La lógica de inventario funciona automáticamente.

Compras

Compra → stock aumenta

Ventas

Venta → stock disminuye

INTEGRACIÓN ENTRE APIs

Actualmente:

- API Principal → usa SQL Server
- API E-Commerce → usa datos Mock

Integración futura:

La API E-Commerce consumirá directamente los endpoints de la API Principal.

Especialmente:

- Productos
- Clientes
- Ventas
- Inventario

AZURE QUEUE STORAGE

El sistema está preparado para integrar Azure Queue Storage.

Objetivo:

- Procesar mensajes
- Manejar pedidos
- Notificaciones automáticas
- Estados de venta

Ejemplo:

Venta → Estado cambiado → Azure Queue → Notificación

PROBLEMAS SOLUCIONADOS

Problemas de conexión en Postman

Se solucionó:

- Ejecutando correctamente localhost
- Configurando puertos
- Probando endpoints desde Swagger

Error de namespaces

Se solucionó:

- Ajustando nombres de proyecto
- Corrigiendo using y namespaces

Error 404 Producto no encontrado

Se solucionó:

- Agregando productos al DatosMock
- Corrigiendo ids

ESTADO ACTUAL DEL PROYECTO

API Principal

Completado:

- CRUD Productos
- CRUD Clientes
- CRUD Proveedores
- Compras
- Ventas
- Inventario
- Estados
- Reportes
- SQL Server
- Swagger

API E-Commerce

Completado:

- Catálogo
- Paginación
- Filtros
- Carrito
- Compra simulada
- Datos Mock
- Swagger
- Postman

MEJORAS FUTURAS

- JWT Authentication
- Roles y permisos
- Docker completo
- Azure Queue real
- Integración frontend
- Integración total entre APIs
- Base de datos compartida
- Validaciones avanzadas
- Logs

CONCLUSIÓN

El proyecto quedó preparado como una solución escalable para un supermercado.

Actualmente permite:

- Gestionar productos
- Gestionar clientes
- Gestionar proveedores
- Registrar compras
- Registrar ventas
- Procesar compras desde carrito
- Generar reportes
- Manejar inventario
- Integrar futuras tecnologías cloud

La estructura fue diseñada para facilitar la integración entre módulos y permitir crecimiento futuro del sistema.